

EventDrivenTradingEngine Portfolio

EventDrivenTradingEngine

2026-04-26

1. Project Overview

EventDrivenTradingEngine is a C++23 event-driven trading simulation. It consumes market ticks from CSV, computes strategy signals, applies risk controls, and updates portfolio state over an event loop.

- Build system: CMake
- Primary language: C++23
- Core code path: `src/`
- Output/result path: `data/`

2. Repository Structure

```
src/  
  data_feed.h / data_feed.cpp  
  strategy.h  / strategy.cpp  
  portfolio.h / portfolio.cpp  
  risk.h      / risk.cpp  
  engine.h    / engine.cpp  
  main.cpp
```

2.1 Architecture Summary

- Data Feed: loads CSV into tick events
- Strategy: computes BUY / SELL / HOLD signals
- Portfolio: tracks cash, position, and equity
- Risk Manager: monitors drawdown and risk states
- Engine: orchestrates the event-driven processing loop

3. Test Results (from data folder)

This section presents the run outputs found in `data/`.

3.1 Output Files

- `data/tr_eikon_eod_data.csv`: market input data
- `data/signals.csv`: per-tick signal, equity, and drawdown log

- data/trades.csv: executed trades
- data/report.png: plotted performance report

3.2 Key Metrics

Metric	Value
Signal rows	1012
Start date	2010-01-04
End date	2014-01-09
Initial net value	10000.00
Final net value	11620.10
Total return	16.2010%
Max drawdown	12.9922%
Min price	27.4357
Max price	100.3000
BUY signals	28
SELL signals	28
HOLD/no-action signals	956
trades.csv rows	0

3.3 Sample Rows

signals.csv (head)

```
Date,Price,Signal,NetValue,Drawdown
2010-01-04,30.5728,----,10000,0
2010-01-05,30.6257,----,10000,0
```

signals.csv (tail)

```
2014-01-08,77.6371,----,11620.1,0.129922
2014-01-09,76.6455,----,11620.1,0.129922
```

trades.csv

File is currently empty (no executed trades in this run).

3.4 Performance Figure

4. C++ Code Appendix (src files)

File: src/data_feed.h

```
#pragma once
#include <string>
#include <vector>

struct Tick {
    std::string date;
```

Event-Driven Trading Engine — Performance Report

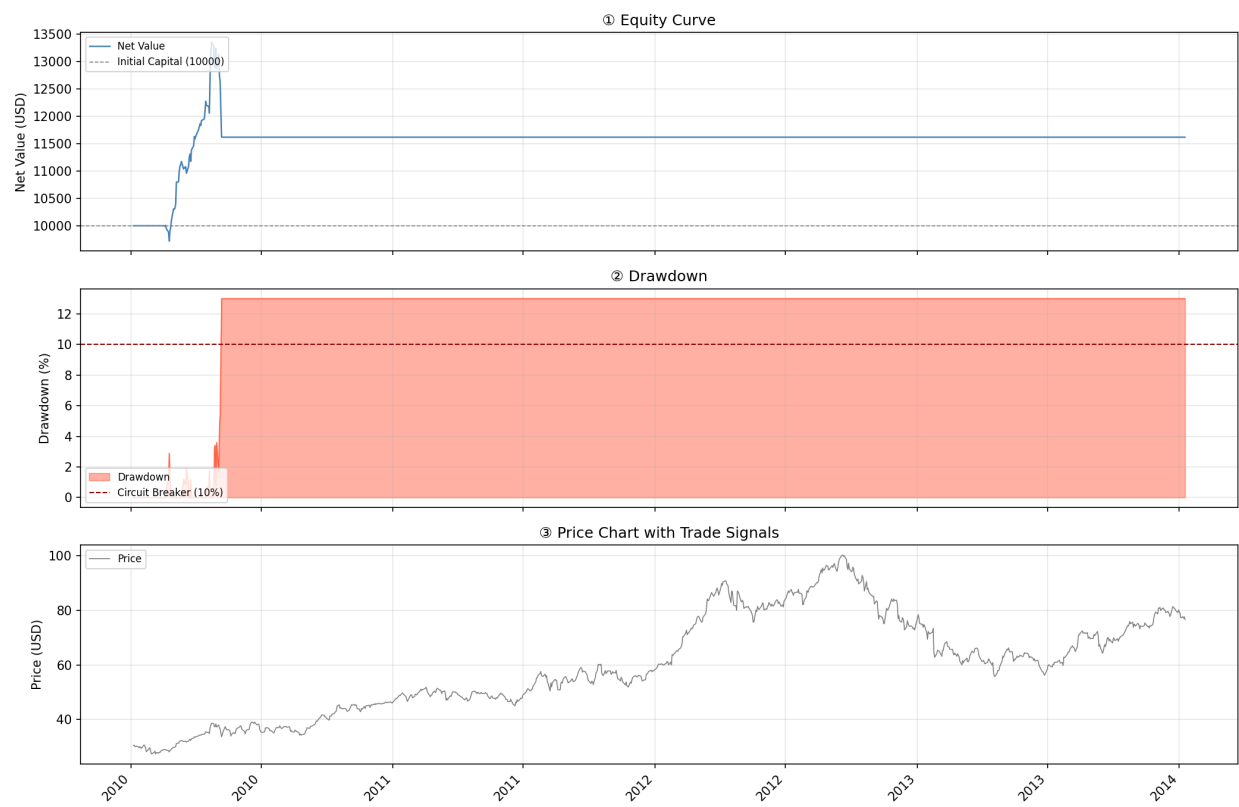


Figure 1: Performance Report

```

    double price;
};

std::vector<Tick> loadCSV(const std::string& filename);

```

File: src/engine.h

```

#pragma once
#include <vector>
#include "data_feed.h"

void runEngine(const std::vector<Tick>& ticks);

```

File: src/portfolio.h

```

#pragma once
#include <string>
#include <vector>

struct Trade {
    std::string date;
    std::string action;
    double price;
    int quantity;
    double commission;
    double cashAfter;
};

struct Portfolio {
    double cash;
    int position;
    std::vector<Trade> tradeLog;
};

Portfolio createPortfolio(double initialCash);

bool executeBuy(Portfolio& portfolio, const std::string& date, double price);

bool executeSell(Portfolio& portfolio, const std::string& date, double price);

double getNetValue(const Portfolio& portfolio, double currentPrice);

```

File: src/risk.h

```

#pragma once
#include <string>

struct RiskManager{
    double peakEquity;
};

```

```

    double drawdown;
    double maxDrawdown;
    bool halted;
    double limit;
};

```

```

RiskManager createRiskManager(double initialEquity, double limit = 0.1);
void updateRisk(RiskManager& riskManager, double currentEquity, const std::string& date);

```

File: src/strategy.h

```

#pragma once
#include <vector>

```

```

enum class Signal {
    BUY,
    SELL,
    HOLD
};

```

```

Signal computeSignal(const std::vector<double> &prices, int shortLen = 5, int longLen = 20);

```

File: src/data_feed.cpp

```

#include "data_feed.h"
#include <fstream>
#include <sstream>
#include <iostream>

```

```

std::vector<Tick> loadCSV(const std::string &filename) {
    std::vector<Tick> ticks;
    std::ifstream file(filename);

    if (!file.is_open()) {
        std::cerr << "Error: could not open " << filename << std::endl;
        return ticks;
    }
}

```

```

std::string line;
std::getline(file, line); // skip the header row

```

```

while (std::getline(file, line)) {
    std::stringstream ss(line);
    std::string date, priceStr;

    std::getline(ss, date, ','); // column 0: Date
    std::getline(ss, priceStr, ','); // column 1: AAPL.O
}

```

```

    if (priceStr.empty())
        continue; // row has no Apple price

    Tick t;
    t.date = date;
    t.price = std::stod(priceStr);
    ticks.push_back(t);
}

return ticks;
}

```

File: src/engine.cpp

```

#include "engine.h"
#include "portfolio.h" // Week 5
#include "risk.h" // Week 6
#include "strategy.h" // Week 4
#include <chrono>
#include <fstream> // std::ofstream CSV
#include <iostream>
#include <thread>

void runEngine(const std::vector<Tick> &ticks) {
    std::cout << " " << ticks.size() << " "
        << std::endl;

    // Ctrl+C
    //
    // Ctrl+C ""Signal
    //
    // - Ctrl+C
    // - SIGINT Signal Interrupt""
    // -
    //
    //
    // "~C" Ctrl+C
    std::cout << " Ctrl+C " << std::endl;
    std::cout << std::endl;

    int cycle = 1; //

    //
    while (true) {
        std::cout << "--- " << cycle << " ---" << std::endl;

        //
        //
        Portfolio portfolio = createPortfolio(10000.0);
    }
}

```

```

RiskManager rm = createRiskManager(10000.0); // Week 6

//
// std::ofstream
// sigCsv, tradeCsv while
// RAII close
std::ofstream sigCsv("data/signals.csv");
std::ofstream tradeCsv("data/trades.csv");

// CSV
// signals.csv Drawdown Week 7
sigCsv << "Date,Price,Signal,NetValue,Drawdown\n";
tradeCsv << "Date,Action,Price,Quantity,Commission,CashAfter\n";

//
// priceHistory tick
// computeSignal
std::vector<double> priceHistory;

double prevPrice = ticks[0].price; // Tick

//
for (int i = 0; i < (int)ticks.size(); i++) {
    double price = ticks[i].price;
    std::string date = ticks[i].date;

    // Week 3
    std::string direction;
    if (price > prevPrice)
        direction = "UP ";
    else if (price < prevPrice)
        direction = "DOWN";
    else
        direction = "FLAT";

    // tick
    priceHistory.push_back(price);

    // tick Week 4
    Signal sig = computeSignal(priceHistory);

    // "BUY " "SELL" / "----"
    std::string sigStr;
    if (sig == Signal::BUY)
        sigStr = "BUY ";
    else if (sig == Signal::SELL)
        sigStr = "SELL";
    else

```

```

sigStr = "----";

// Week 6
//
//
// 1 BUY
// 2
//
// BUY SELL
// ""
//
if (rm.halted) {
    // 2
    if (portfolio.position > 0) {
        bool sold = executeSell(portfolio, date, price);
        if (sold) {
            std::cout << " [] " << date << " "
                << portfolio.tradeLog.back().quantity << " @ " << price
                << std::endl;
        }
    }
    // 1 BUY
} else {
    //
    if (sig == Signal::BUY) {
        bool traded = executeBuy(portfolio, date, price);
        if (traded) {
            std::cout << " [] " << portfolio.tradeLog.back().quantity
                << " @ " << price
                << " : " << portfolio.tradeLog.back().commission
                << " : " << portfolio.cash << std::endl;
        }
    } else if (sig == Signal::SELL) {
        bool traded = executeSell(portfolio, date, price);
        if (traded) {
            std::cout << " [] " << portfolio.tradeLog.back().quantity
                << " @ " << price
                << " : " << portfolio.tradeLog.back().commission
                << " : " << portfolio.cash << std::endl;
        }
    }
}

//
// getNetValue = +
double netValue = getNetValue(portfolio, price);

// Week 6

```



```

// netValue
// updateRisk
updateRisk(rm, netValue, date);

//
std::cout << date << " " << direction << " " << sigStr
           << " price: " << price << " net_value: " << netValue
           << " drawdown: " << rm.drawdown * 100.0 << "%"
           << (rm.halted ? " []" : "") << std::endl;

// Drawdown
sigCsv << date << "," << price << "," << sigStr << "," << netValue << ","
        << rm.drawdown << "\n";

prevPrice = price; //

// 50
std::this_thread::sleep_for(std::chrono::milliseconds(50));
}

// trades.csv
// for Range-based for loopC++11
// for (const Trade& t : portfolio.tradeLog) { ... }
// const Trade& t Trade t
// tradeLog
for (const Trade &t : portfolio.tradeLog) {
    tradeCsv << t.date << "," << t.action << "," << t.price << ","
             << t.quantity << "," << t.commission << "," << t.cashAfter
             << "\n";
}

//
std::cout << std::endl;
std::cout << " " << portfolio.tradeLog.size() << " "
           << std::endl;
for (const Trade &t : portfolio.tradeLog) {
    std::cout << " " << t.date << " " << t.action << " " << t.quantity
              << " "
              << " @ " << t.price << " : " << t.commission
              << std::endl;
}

//
// ticks.back() vector
double finalNetValue = getNetValue(portfolio, ticks.back().price);
std::cout << ": " << finalNetValue
          << " : " << (finalNetValue - 10000.0) << std::endl;

```

```

// Week 6
std::cout << ": " << rm.maxDrawdown * 100.0 << "%"
            << (rm.halted ? " " : " ")
            << std::endl;

std::cout << std::endl;
std::cout << " data/signals.csv "
            << "data/trades.csv..."
            << std::endl;
std::cout << std::endl;

cycle++; //
}
}

```

File: src/main.cpp

```

#include "data_feed.h"
#include <iostream>
#include "engine.h"

int main() {
    std::cout << "Event-Driven Trading Engine -- Market Data Feed" << std::endl;
    std::cout << "-----" << std::endl;

    std::vector<Tick> ticks = loadCSV("data/tr_eikon_eod_data.csv");

    if (ticks.empty()) {
        std::cout << "No data loaded. Check that data/tr_eikon_eod_data.csv exists."
                  << std::endl;
        return 1;
    }

    std::cout << "Loaded " << ticks.size() << " AAPL.O ticks." << std::endl;

    runEngine(ticks);
    return 0;
}

```

File: src/portfolio.cpp

```

#include "portfolio.h"
#include <cmath>

const double COMMISSION_RATE = 0.001;

Portfolio createPortfolio(double initialCash){
    Portfolio p;
}

```

```

    p.cash = initialCash;
    p.position = 0;
    return p;
}

bool executeBuy(Portfolio& portfolio, const std::string& date, double price){
    if(portfolio.position > 0) {
        return false;
    }
    int shares = static_cast<int>(std::floor(portfolio.cash / (price * (1.0 + COMMISSION_RATE))));
    if(shares <= 0) {
        return false;
    }
    double cost = shares * price;
    double commission = cost * COMMISSION_RATE;
    portfolio.cash -= (cost + commission);
    portfolio.position += shares;
    portfolio.tradeLog.push_back({date, "BUY", price, shares, commission, portfolio.cash});
    return true;
}

bool executeSell(Portfolio& portfolio, const std::string& date, double price){
    if(portfolio.position <= 0) {
        return false;
    }
    int shares = portfolio.position;
    double revenue = shares * price;
    double commission = revenue * COMMISSION_RATE;
    portfolio.cash += (revenue - commission);
    portfolio.position -= shares;
    portfolio.tradeLog.push_back({date, "SELL", price, shares, commission, portfolio.cash});
    return true;
}

double getNetValue(const Portfolio& portfolio, double currentPrice){
    return portfolio.cash + portfolio.position * currentPrice;
}

```

File: src/risk.cpp

```

#include "risk.h"
#include <iostream>
#include <algorithm> // std::max

RiskManager createRiskManager(double initialEquity, double limit){
    RiskManager rm;
    rm.peakEquity = initialEquity;
    rm.drawdown = 0.0;
}

```

```

    rm.maxDrawdown = 0.0;
    rm.halted = false;
    rm.limit = limit;
    return rm;
}

void updateRisk(RiskManager& riskManager, double currentEquity, const std::string& date){
    riskManager.peakEquity = std::max(riskManager.peakEquity, currentEquity);
    if (riskManager.peakEquity > 0.0) {
        riskManager.drawdown = (riskManager.peakEquity - currentEquity) / riskManager.peakEquity;
    } else {
        riskManager.drawdown = 0.0;
    }
    riskManager.maxDrawdown = std::max(riskManager.maxDrawdown, riskManager.drawdown);
    if(riskManager.drawdown >= riskManager.limit && !riskManager.halted) {
        riskManager.halted = true;
        std::cout << ": " << date
            << " : " << riskManager.drawdown * 100 << "% : "
            << riskManager.maxDrawdown * 100 << "%" << std::endl;
    }
}

```

File: src/strategy.cpp

```

#include "strategy.h"
#include <numeric> // std::accumulate

//
// smaSimple Moving Average
//
//
// prices
// len      len
//
// len
//
// prices = {30.0, 31.0, 29.0, 32.0, 30.5}len = 3
//   3   {29.0, 32.0, 30.5}
//   = 91.5 = 30.5
//
// std::accumulate
//   std::accumulate(, , )
//
//   prices.end() - len len
//
static double sma(const std::vector<double> &prices, int len) {
    // prices.end() - len len
    // prices.end()

```

```

    double sum = std::accumulate(prices.end() - len, prices.end(), 0.0);
    return sum / len; //
}

//
// computeSignal
//
Signal computeSignal(const std::vector<double> &prices, int shortLen,
                    int longLen) {

    // 1
    //
    // 20 HOLD
    if ((int)prices.size() < longLen) {
        return Signal::HOLD;
    }

    // ""
    double shortNow = sma(prices, shortLen); // 5
    double longNow = sma(prices, longLen); // 20

    // 2
    // "" longLen+1
    // HOLD
    if ((int)prices.size() < longLen + 1) {
        return Signal::HOLD;
    }

    // ""
    // prices ""
    // std::vector prices [0, size-1)
    std::vector<double> prevPrices(prices.begin(), prices.end() - 1);
    double shortPrev = sma(prevPrices, shortLen); // 5
    double longPrev = sma(prevPrices, longLen); // 20

    //
    // Golden Cross BUY
    // shortPrev <= longPrev
    // shortNow > longNow
    if (shortPrev <= longPrev && shortNow > longNow) {
        return Signal::BUY;
    }

    // Death Cross SELL
    // shortPrev >= longPrev
    // shortNow < longNow
    if (shortPrev >= longPrev && shortNow < longNow) {
        return Signal::SELL;
    }
}

```

```
}  
  
//  HOLD  
return Signal::HOLD;  
}
```